✦ N O V A ✦

Programming Language

Language Traits & Type System

Design Document · v0.1

Foundational Reference for Compiler & Runtime

Units as types. Tensors as citizens. Parallelism as default. Clarity as law.

01 Overview — What This Document Is

This document is the canonical reference for NOVA's language traits and type system. It defines what features NOVA borrows from C, Rust, Python, and Java; how those traits are adapted; and how NOVA's own type system is constructed on top of them.

Every compiler engineer, contributor, and language designer working on NOVA should read this document before writing any code. It is the contract between the language design and its implementation.

> *This document is Step 1 of the NOVA compiler roadmap. The lexer, parser, type checker, and LLVM backend all depend on the decisions made here.*

02 Multi-Language Compiler Architecture

NOVA's compiler is deliberately split across multiple implementation languages. Each phase of compilation uses the language best suited to that phase's requirements. This is not a compromise — it is a strength. The compiler is a system of systems, and each subsystem should be written in the language that makes it most correct, most maintainable, and most performant.

C Family	Rust	Python	Java
Raw performance Lexer / Parser Memory model	Safety & ownership Trait system Compiler backend	Readable syntax Scripting feel STDLIB style	Interfaces Class hierarchy OOP patterns

How the phases divide

The compiler pipeline has five major phases. Each phase is assigned a primary implementation language based on the nature of the work:

Phase	Component	Language	Status	Depends On
P1	Lexer (tokenizer)	`C`	**Next**	—
P1	Parser (AST builder)	`C`	**Next**	Lexer
P2	Unit resolver	`Rust`	**Planned**	Parser / AST
P2	Type checker	`Rust`	**Planned**	Unit resolver
P3	Semantic analyser	`Rust`	**Planned**	Type checker
P3	Interface validator (OOP)	`Java`	**Future**	Semantic analyser

Phase	Component	Language	Status	Depends On
P4	Autodiff + tensor lowering	`Rust`	**Future**	Type checker
P4	Parallel scheduler	`Rust/C`	**Future**	Semantic analyser
P5	LLVM IR emitter	`Rust`	**Future**	All above
P5	Standard library (cosmos.*)	`Python`	**Future**	LLVM IR
P5	REPL / scripting shell	`Python`	**Future**	Parser

Why C for the Lexer and Parser?

The lexer and parser are the hottest path in the compiler. They are run on every source file, every keystroke in the LSP, and every REPL input. C gives us direct memory control, zero allocation overhead, and predictable performance. The lexer will be a hand-written finite automaton in C. The parser will be a hand-written recursive descent parser in C, producing an AST that Rust can consume via a C FFI boundary.

Why Rust for the type system and backend?

Rust's ownership model makes the type checker and unit resolver safe by construction. A type error in the type checker should never produce a use-after-free or a dangling pointer. Rust's trait system maps cleanly onto NOVA's own trait/interface design. The LLVM Rust bindings (inkwell) give us a safe, expressive API for IR emission.

Why Java for OOP / interface validation?

NOVA borrows Java's interface and class hierarchy model for its constellation (module) system and its OOP patterns. The interface validator — the compiler phase that checks that a type correctly implements an interface — is prototyped in Java because Java's own reflection and type hierarchy tools make that logic easiest to reason about and test. It will eventually be rewritten in Rust for the production compiler, but Java is used first for correctness.

Why Python for the standard library and REPL?

The cosmos.* and nova.* standard constellations involve a large amount of scientific and general-purpose logic that benefits from Python's rapid iteration, rich numerical ecosystem, and readable code. The REPL and scripting shell also benefit from Python's dynamic feel. Python standard library implementations will be transpiled or wrapped into NOVA-native code as the compiler matures.

03 Language Traits — What NOVA Borrows and How

NOVA is not a copy of any single language. It borrows specific, well-chosen traits from each of its inspirations, adapts them to the NOVA philosophy, and in several cases extends or combines them in ways that none of the source languages do. The table below is the canonical list of borrowed traits.

Trait / Feature	Borrowed From	How NOVA Uses It	NOVA Keyword / Syntax
Manual memory (opt-in)	C	Scientists can opt into raw allocation with `raw` blocks for performance-critical paths	`raw { ... }` block
Pointer arithmetic (opt-in)	C	Available in `unsafe` regions for low-level astronomy data buffers and tensor storage	`unsafe { ptr }` block
Struct layout control	C	Structs can specify packed/aligned layout for FFI and hardware interfacing	`struct @packed`, `struct @align(n)`
Deterministic performance	C	Compilation to native code with no GC pauses; unit types erased at runtime	`nova build --opt`
C FFI interop	C	NOVA can call any C function directly; C can call NOVA via exported missions	`extern "C" mission name()`
Ownership & borrowing	Rust	NOVA values are owned by default; borrowing uses `&` notation; lifetimes are inferred	`let x = &value` (borrow)
Trait system	Rust	NOVA traits define shared behaviour; types implement traits; trait bounds on missions	`trait Printable { ... }`
Result / Option types	Rust	`Option[T]` for nullable; `Result[T, E]` for fallible; `?` for propagation	`let x = op()?`
Enum with data (ADT)	Rust	NOVA enums can hold data per variant, enabling algebraic data types and safe modelling	`enum Signal { Some(T), None }`
Memory safety guarantees	Rust	The compiler proves memory safety at compile time; no dangling pointers outside `unsafe`	Enforced by type checker
Pattern matching (exhaustive)	Rust	`match` is exhaustive; all variants must be handled or the compiler errors	`match x { ... }`
Readable, flat syntax	Python	NOVA syntax reads like English; no semicolons; indentation is syntactically meaningful	Indentation + `{}`
Named arguments	Python	Mission arguments can be passed by name in any order; improves readability in science	`delta_v(isp=311.0[s], ...)`

Trait / Feature	Borrowed From	How NOVA Uses It	NOVA Keyword / Syntax
String interpolation	Python	f-string style with format specifiers and unit conversion inline	`"value = {v as [km/s]:.2}"`
REPL / interactive use	Python	nova-repl provides a Python-like interactive session with type and unit display	`nova-repl`
Duck-typed stdlib feel	Python	Standard constellations use flexible, readable APIs; less ceremony than Java	`cosmos.stats.pearson(a, b)`
Interface contracts	Java	Constellations export interfaces; types implement them; the compiler validates conformance	`interface Measurable { ... }`
Class hierarchy (structs)	Java	NOVA structs can extend other structs (single inheritance); interfaces allow multiple impl	`struct B extends A { ... }`
Checked exceptions → Result	Java	Java's checked exception philosophy is kept but expressed as `Result[T, E]` types	`Result[Float[m/s], UnitError]`
Namespace organisation	Java	Constellations mirror Java packages; hierarchical, dot-separated, explicit import	`absorb cosmos.stats.{ pearson }`
Explicit visibility	Java	All missions and types are private by default; must be `export`ed to be public	`export mission foo() → T`

04 NOVA Type System

NOVA's type system is its most distinctive engineering contribution. It extends Hindley-Milner type inference with three novel dimensions: physical unit vectors, tensor shape types, and AI-native primitives. All three are fully resolved at compile time with zero runtime overhead.

4.1 Primitive Types

Type	Example	Compile-time Check	Inspired By
`Int`	`42`	Overflow checked in debug mode	C / Rust
`Float`	`3.14`	IEEE-754 double	C / Rust
`Float[m/s]`	`9.8[m/s²]`	Unit dimension checked at compile time	F# Units of Measure
`Bool`	`true, false`	Cannot coerce to Int (no implicit cast)	Rust

Type	Example	Compile-time Check	Inspired By
`Char`	`'A'`	Unicode scalar value	Rust
`Str`	`"hello"`	Immutable UTF-8 string	Python / Rust
`Void`	—	No return value	C
`Never`	`panic(...)`	Function that never returns	Rust

4.2 Unit Type System

Every numeric value in NOVA may carry a physical unit as part of its type. Units are not wrappers or runtime tags — they are compile-time type parameters that track the SI dimension vector (length, mass, time, current, temperature, amount, luminosity) of each value.

> *Unit arithmetic is fully automatic. Multiplying Float[kg] by Float[m/s²] yields Float[N]. The compiler knows that kg·m/s² = N from the SI dimension table.*

SI Dimension Vector

Every unit type carries a 7-tuple exponent vector [L, M, T, I, Θ, N, J] over the rationals. Examples:

```
Float[m]       →  [ 1,  0,  0,  0,  0,  0,  0]   -- length
Float[kg]      →  [ 0,  1,  0,  0,  0,  0,  0]   -- mass
Float[m/s]     →  [ 1,  0, -1,  0,  0,  0,  0]   -- velocity
Float[m/s²]    →  [ 1,  0, -2,  0,  0,  0,  0]   -- acceleration
Float[N]       →  [ 1,  1, -2,  0,  0,  0,  0]   -- force  (kg·m/s²)
Float[J]       →  [ 2,  1, -2,  0,  0,  0,  0]   -- energy (kg·m²/s²)
Float[Pa]      →  [-1,  1, -2,  0,  0,  0,  0]   -- pressure (kg/(m·s²))
```

Unit Inference Rules

The type checker applies the following rules automatically during type inference:

```
a[U] * b[V]   →   result[(U + V)]   -- vector addition of exponents
a[U] / b[V]   →   result[(U - V)]   -- vector subtraction
a[U] + b[V]   →   ONLY if U == V   -- identical dimension required
a[U] - b[V]   →   ONLY if U == V   -- identical dimension required
a[U] ^ n      →   result[(U * n)]   -- scalar multiplication of exponents
```

Custom Units

```
unit parsec    = 3.086e16[m]        -- Float[parsec] is Float[m] with scale factor
unit M_sun     = 1.989e30[kg]       -- stellar mass unit
unit AU        = 1.496e11[m]        -- astronomical unit
unit eV        = 1.602e-19[J]       -- electron volt
```

Unit Conversion

```
let v = 9800.0[m/s]
```

```
  let v_km = v as [km/s]              -- explicit conversion, type becomes
Float[km/s]
  transmit("{v as [km/s]:.3} km/s")  -- inline conversion in string interpolation
```

4.3 Tensor Type System

Tensors are first-class types in NOVA. The type of a tensor encodes its element type (which may include a unit), its number of dimensions (rank), and optionally its concrete shape. Shape errors are caught at compile time.

Type	Example	Compile-time Check	Inspired By
`Tensor[Float, 3]`	`shape=(n,m,k)`	Rank enforced; matmul shape checked	PyTorch / JAX
`Tensor[Float[eV], 1]`	`spectral energy array`	Unit + rank both enforced	NOVA original
`Tensor[Int, 2]`	`token ID matrix`	Integer tensor; no unit	PyTorch
`Tensor[Float, (4,4)]`	`4×4 rotation matrix`	Concrete shape; indexing checked	NOVA original

> *Shape inference: where concrete shapes are known at compile time, shape mismatches in matmul (@) and einsum are errors, not runtime panics.*

4.4 Trait System

NOVA's trait system is inspired by Rust traits but simplified for scientific programmers. A trait defines a set of missions that a type must implement. Traits can have default implementations. Mission signatures can include trait bounds.

```
  trait Measurable {
    mission measure(self) → Float[kg]
  }
```

```
  trait Displayable {
    mission display(self) → Str
    mission display_with_unit(self, unit: UnitSpec) → Str   -- default impl allowed
  }
```

```
  -- Mission with trait bound: T must implement both traits
  mission report[T: Measurable + Displayable](obj: T) → Void {
    transmit("{obj.display()} weighs {obj.measure()}")
  }
```

> *Unlike Rust, NOVA does not require explicit lifetime annotations in most cases. The borrow checker infers lifetimes using a simplified region-based model.*

4.5 Interface System (Java-inspired)

Constellations (modules) export interfaces. Interfaces define the public contract that a constellation's types must satisfy. This is distinct from traits — interfaces are module-level contracts, traits are type-level contracts.

```
interface Optimiser {
  mission step(params: Tensor[Float, 1], grad: Tensor[Float, 1]) → Tensor[Float, 1]
  mission reset(self) → Void
}
```

```
struct AdamW implements Optimiser {
  var lr  : Float = 3e-4
  var wd  : Float = 0.01
  ...
}
```

4.6 Ownership and Memory Model (Rust-inspired)

NOVA values are owned by default. Ownership can be transferred (move) or shared by borrowing. The rules are:

- Each value has exactly one owner at any time.
- Ownership transfers on assignment or mission call by default (move semantics).
- A value can be borrowed immutably (&T) any number of times simultaneously.
- A value can be borrowed mutably (&mut T) at most once, with no simultaneous borrows.
- Borrows are checked at compile time. No use-after-free. No dangling pointers.

> *Scientific programmers do not need to think about ownership in typical usage. The common pattern — create a tensor, pass it to a mission, use the result — involves no manual lifetime management. Ownership rules only surface at the boundary of performance-critical or FFI code.*

05 Type Inference

NOVA uses a Hindley-Milner style unification algorithm extended to handle unit dimension vectors, tensor ranks, and trait bounds simultaneously. The programmer writes very few type annotations — the compiler infers them.

5.1 Inference examples

```
let mass      = 5.972e24[kg]          -- inferred: Float[kg]
let radius    = 6.371e6[m]             -- inferred: Float[m]
let g         = 6.674e-11[N·m²/kg²]   -- inferred: Float[N·m²/kg²]
let escape_v  = sqrt(2.0 * g * mass / radius)
                                     -- inferred: Float[m/s]  (unit arithmetic)
```

```
let stars = read_fits(catalog)
  |> pipeline [
      filter(luminosity_class == "V"),
      map(s → { mass: s.mass_solar, lum: log10(s.luminosity_solar) }),
    ]
-- stars inferred as: Array[{ mass: Float[M_sun], lum: Float }]
```

5.2 Error messages

Type errors in NOVA are written for scientists and engineers. A unit mismatch names the physical dimensions involved:

```
let wrong = 1.0[kg] + 1.0[m]
```

```
error: cannot add Float[kg] and Float[m]
       dimension [M] (mass) + [L] (length) — incompatible
       hint: did you mean to scale one of these, or is this a logic error?
       --> src/main.nv:12:3
```

06 Pattern Matching

NOVA's match expression is exhaustive (Rust-inspired). All variants of an enum must be handled; the compiler errors if any case is missing. Match arms use the => syntax and can include guards.

```
match detection {
  Some(d) if d.snr > 12.0  =>  confirm(d)
  Some(d)                  =>  flag_for_review(d)
  None                     =>  log_missing()
}
```

```
match result {
  Ok(value)  =>  transmit("Got: {value}")
  Err(e)     =>  transmit("Error: {e.message}")
}
```

> *Partial match (not covering all variants) is a compile-time error, not a warning. This is a deliberate safety decision: in scientific code, an unhandled case usually means an unhandled physical scenario.*

07 Next Steps — What Gets Built After This Document

This document is Step 1. Once it is reviewed and agreed upon, the following steps proceed in order:

Step	Deliverable	Description	Language
02	Project directory structure	Canonical folder layout for the compiler, stdlib, tests, and toolchain	— (scaffold only)
03	Lexer	Tokenises .nv source into the token stream; hand-written FA	C
04	Token spec doc	Full list of NOVA tokens with regex/rules for each	— (document)
05	Parser skeleton	Recursive descent; produces typed AST nodes	C
06	AST definition	C structs for every AST node type; FFI-safe layout	C → Rust FFI
07	Unit resolver	Tracks SI dimension vectors; implements unit arithmetic rules	Rust
08	Type checker stub	HM unification with unit and tensor extensions	Rust